

FreeRTOS notes

Task Management in FreeRTOS

A task is nothing exotic - a C function, a private stack, and one small struct the scheduler uses to keep track of it. Everything below is really just different views of that one idea.

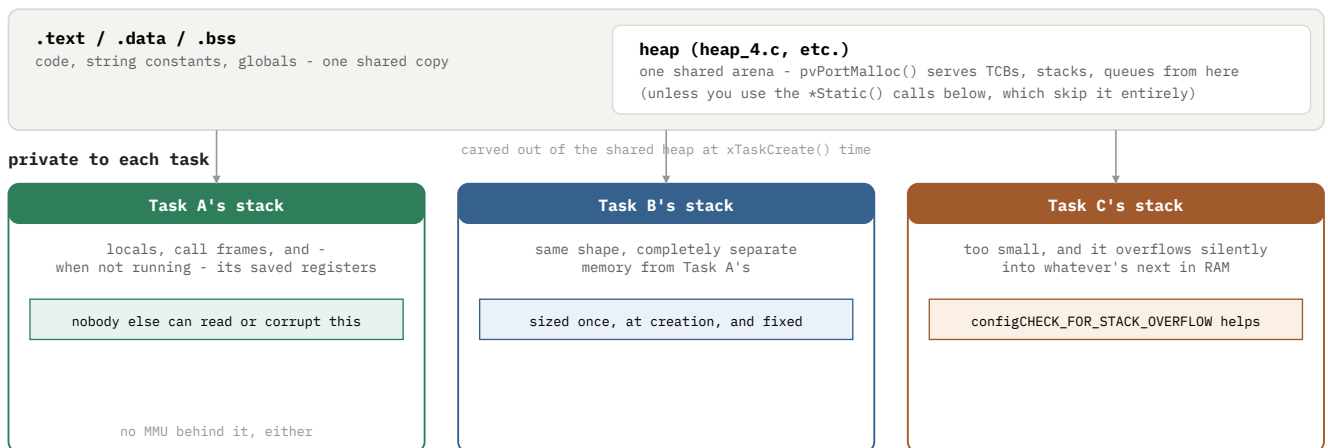
contents

1. A task in memory
2. Task states, lifecycle & types
3. The Task Control Block
4. Task scheduling & scheduling queues
5. The scheduling algorithm

1 A task in memory

Creating a task with `xTaskCreate()` doesn't carve out a new address space - FreeRTOS has no MMU to do that with. It hands the new task exactly one thing that's genuinely its own: a stack. Everything else - the code, the globals, the heap those `xTaskCreate()` calls come out of - is one shared region every task in the system reads and writes equally.

one flat address space



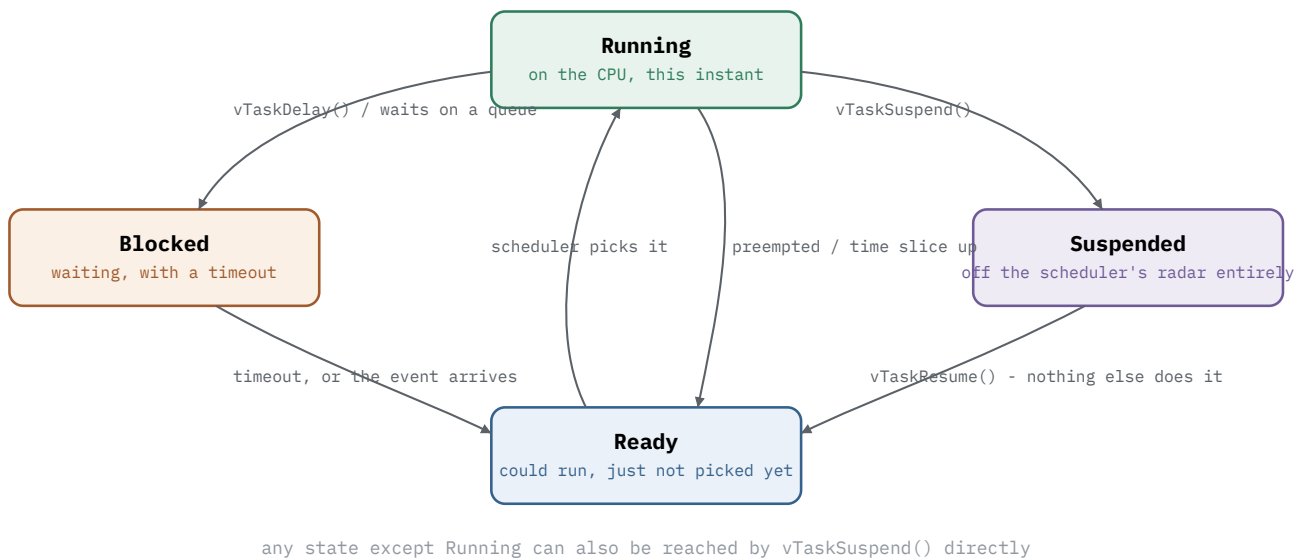
The TCB itself isn't shown here on purpose - it's a separate, small allocation that just remembers where each of these stacks is. That's section 3.

The trade being made here is the same one behind every design choice in this doc: no MMU means no per-task isolation, but it also means no page tables to walk and no translation overhead on every memory access - one less source of "how long could this possibly take" in a system that has to answer that question.

2 Task states, lifecycle & types

States - what a live task is doing right now

Four states cover every live task in the system. Only one task can be *Running* at a time per core; everything else is Ready, Blocked, or Suspended.



Lifecycle - birth to death, once each

The state diagram above is the loop a task lives in. It sits inside a longer, one-way story: something creates the task, and eventually something deletes it.



A task can't free its own stack while standing on it - deletion just marks it gone; the Idle task does the actual cleanup the next time it runs.

Types

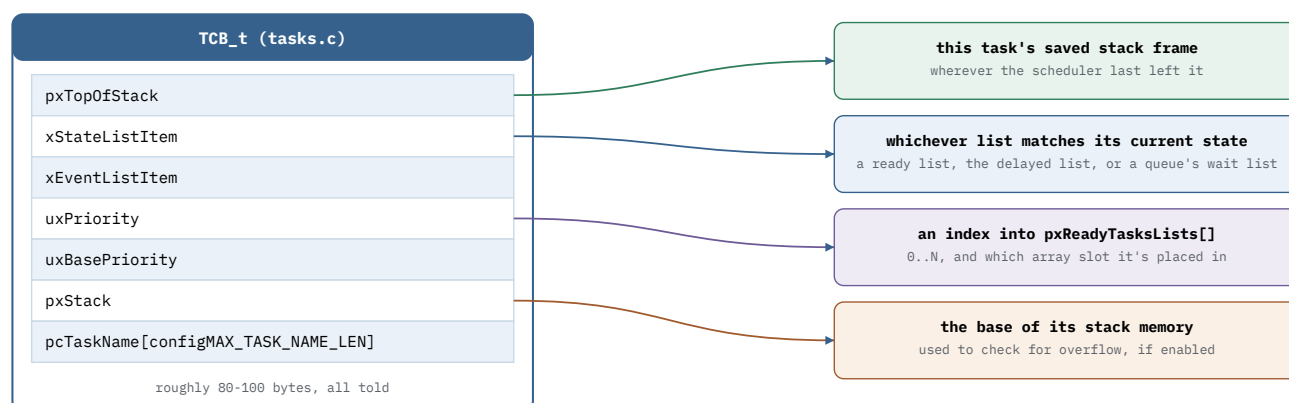
"Type" splits two ways in FreeRTOS - by how a task's memory was obtained, and by who created it.

BY ALLOCATION	DYNAMIC	STATIC
Call	<code>xTaskCreate()</code>	<code>xTaskCreateStatic()</code>
TCB and stack come from	the heap, via <code>pvPortMalloc()</code>	buffers <i>you</i> declare and pass in
Needs	a heap implementation linked in (<code>heap_4.c</code> , etc.)	<code>configSUPPORT_STATIC_ALLOCATION</code> - no heap needed at all

BY ORIGIN	IDLE TASK	TIMER SERVICE TASK	APPLICATION TASKS
Created by	the kernel, inside <code>vTaskStartScheduler()</code>	the kernel, if <code>configUSE_TIMERS</code> is 1	you, explicitly
Priority	lowest - <code>tskIDLE_PRIORITY</code>	<code>configTIMER_TASK_PRIORITY</code>	whatever you assign
Job	run when nothing else can; reclaim deleted tasks' memory	run software-timer callbacks	your <code>led_1s</code> , <code>led_2s</code> , <code>led_3s</code> , etc.

3 The Task Control Block

Every fact the scheduler knows about a task lives in one struct - `TCB_t` , defined in `tasks.c` . It's small on purpose: a stack pointer, a couple of list links, a priority, a name.



Every non-trivial FreeRTOS operation - creating, deleting, blocking, switching - boils down to reading or rewriting a handful of these fields.

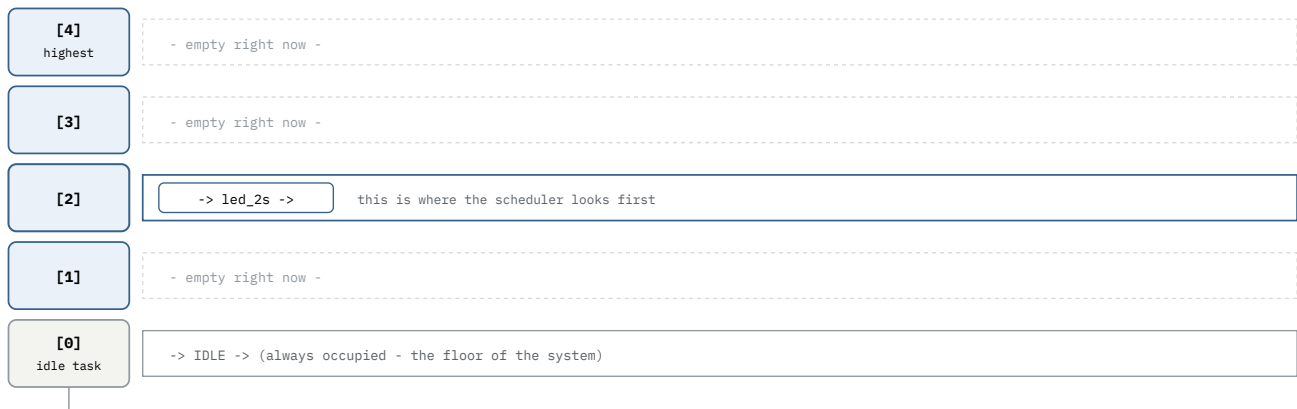
Two of these are worth dwelling on. `pxTopOfStack` is a single pointer, yet it's the entire reason a context switch works at all - it's the address the next switch reads to know exactly where this task's registers were left. And `xStateListItem` is what makes "state" real: a task isn't Ready or Blocked because of a flag somewhere - it's Ready or Blocked because this list item is currently threaded into one linked list rather than another.

4 Task scheduling & scheduling queues

"Queue" here doesn't mean a FIFO - it means the actual data structure the scheduler walks to decide who runs next. FreeRTOS keeps one list per priority level, plus a separate list for anyone waiting on a timeout.

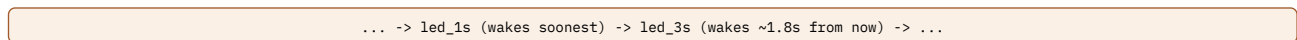
`pxReadyTasksLists[configMAX_PRIORITIES]`

one linked list per priority level - an array, indexed directly by priority number



`pxDelayedTaskList`

a different list entirely - kept sorted by wake tick, not by priority



A task moves between these lists constantly, but it's always in exactly one of them (Running is the special case - it's in a ready list too, just also currently executing).

The actual scheduling event is almost anticlimactic: a tick interrupt or a task blocking/waking pends a switch, that switch calls `vTaskSwitchContext()`, and that function's whole job is "find the highest-priority non-empty entry in `pxReadyTasksLists[]` and make it `pxCurrentTCB`." *How* it finds that entry quickly is its own topic - the next section.

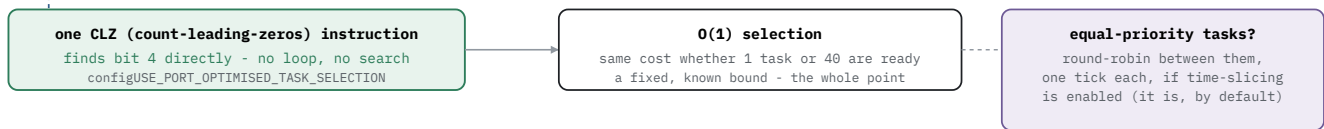
5 The scheduling algorithm

Fixed-priority preemptive, full stop. The highest-priority Ready task always runs. A priority-9 task makes zero progress while any priority-10 task is Ready - there's no

fairness mechanism, and none is wanted. Tasks that share a priority round-robin between each other via time-slicing, if `configUSE_TIME_SLICING` is on.

finding "highest occupied priority" without searching

one bit per priority level, set when that level has ≥ 1 ready task:



contrast this with Linux's CFS: a red-black tree ranked by `vruntime`, chasing fairness across unrelated workloads. FreeRTOS never has that goal - "most important thing runs" beats "everything gets a fair share."

The bitmap lives in a variable the port layer maintains as tasks enter and leave ready lists - it's kept in sync incrementally, never rebuilt from scratch.

Where the actual switch happens. None of this section moves a single register - it only decides *which* TCB becomes `pxCurrentTCB`. Loading that task's registers back onto the core is `PendSV_Handler`'s job, not the scheduler's - covered in the `SysTick` / `PendSV` / `SVC` notes.

FreeRTOS notes · grounded in `FreeRTOS-Kernel/tasks.c` and `portable/GCC/ARM_CM3/port.c`